Theories of Computation

Revision Lecture
May 9th 2023

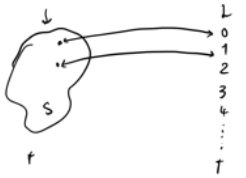
Mathematical techniques

1. Induction and course-of-value induction
   - base case : prove property for 0
   - inductive step : suppose property for a fixed n
     prove property for n+1

↳ this has proved property for all $n \in \mathbb{N}$

suppose property for $0 \leq k \leq n$
prove property for $n+1$

2. Countable and uncountable sets
   → There are some bit strings that are not computable
   → There are some irregular languages



Languages and automata

1. Regular expressions
2. DFA → partial DFA & how to make it total with error state
3. NFA and ε-NFA
   → how to remove ε-transitions & to determinize it
4. Kleene Theorem: reg exp → ε-NFA → NFA → DFA
   a language is recognized by a reg exp iff it is accepted by a DFA
   → how to convert a regexp into a DFA
5. Irregularity (no Pumping Lemma) — Assume it is regular by K.Th. that there is a DFA recog it would lead to infinitely many states — contradiction
6. Test equivalence of automata
7. Automaton for complement — A for language L, A' for L^c
8. Minimality & minimization — accept same language
   ↳ prove not minimal: 2 equivalent states
   prove minimal: produce the proof: all states inequivalent & reachable & hopeful
9. Context-free languages
   → Chomsky normal form
   → Derivation tree & leftmost derivation (right)
10. Ambiguity → prove ambiguity: give a word that has 2 distinct derivation trees

Complexity

1. Complexity of algorithm vs complexity of problems

2. Lower and upper bounds

3. O, Ω, Θ notation ← $f(n) \leq n^3 + 7n^2 + 4$

4. Polynomial and exponential complexity

5. NP-problems, 2 definitions
   question whether P = NP
   - NDTM running in polytime
   - 2 steps (ND)
     - guessing a candidate (certificate)
     - checking correctness of this candidate in polytime (D)

6. NP-completeness and SAT
   → any pb in NP can be reduced in polytime to SAT

Turing machines

1. How to execute Turing machines

2. Design TM for simple tasks
   → macros

3. Fancy TM ⟹ extended alphabet
   → 2 tape
   ↳ can be simulated by an ordinary TM with polynomial overhead
   ↳ does not affect the polynomial complexity but linear, quadratic, etc.

4. Church's thesis:
   any function on words that can be computed algorithmically can be computed by a TM

Decidability

1. Decision problem = answer is yes or no

2. Decidable & semi-decidable problems
   (decison proc. ↳ semi-decision proc
   → a pb is decidable if both it and its complement are semidecidable

3. Primitive recursive functions
   → if implementable in Prim. Java

4. Ackermann function : not prim. rec.

5. Halting Theorem:
   whether a program halts or not is undecidable

6. Rice's Theorem: (I/O behavior)
   any semantic property of code that sometimes hold and sometimes fails to hold is undecidable

7. To show a problem decidable
   → write a program (or program sketch) that decides it

8. To show a problem undecidable
   → reduce the halting problem to it
   → apply Rice's theorem